Student Name :Shaik Mubarak
Reg No:192425194

CO4 – AT-1
MLA0201-FUNDAMENTALS OF MACHINE LEARNING

Annexure A
Coding Challenge

Question 1: Bayesian Network for Disease Prediction
A healthcare organization wants to predict whether a patient has Diabetes based on the symptoms Obesity and High Blood Sugar.
Coding Challenge:
Write a Python program to:
• Construct a Bayesian Network using the given variables.
• Define suitable Conditional Probability Tables (CPTs).
• Perform probabilistic inference to predict the probability of Diabetes when both symptoms are present.
• Display the predicted probability.

Question 2: Hidden Markov Model for Weather Prediction
A weather forecasting agency records daily observations as Sunny, Cloudy, and Rainy. The actual weather conditions are hidden and need to be predicted.
Coding Challenge:
Write a Python program to:
• Implement a Hidden Markov Model (HMM).
• Define transition and emission probabilities.
• Predict the hidden weather states for a given sequence of observations.
• Print the predicted sequence of hidden states.

Question 3: Markov Random Field for Image Denoising
A computer vision company wants to remove noise from grayscale images using a Markov Random Field (MRF).
Coding Challenge:
Write a Python program to:
• Represent a small image as a graph where each pixel is a node.
• Model neighboring pixel relationships using an undirected graph.
• Simulate one iteration of image smoothing based on neighboring pixel values.
• Display the original and updated pixel values.

Question 4: Conditional Random Field for Named Entity Recognition
A customer support system needs to identify Customer Names, Order IDs, and Product Names from support emails.
Coding Challenge:
Write a Python program to:
• Create sequential training data with word-level features.
• Train a Conditional Random Field (CRF) model.
• Predict entity labels for a new sentence.
• Display the predicted labels.

Question 5: Bayesian Network Learning from Data
An insurance company wants to learn probabilistic relationships among Age, Income, Vehicle Type, and Insurance Claim using historical customer data.
Coding Challenge:
Write a Python program to:
• Load a dataset using Pandas.
• Learn the Bayesian Network structure from the data.
• Perform inference to predict the probability of an insurance claim for a new customer.
• Display the learned network structure and prediction results.

QUESTION 1: BAYESIAN NETWORK FOR DISEASE PREDICTION

Problem Understanding

The objective of this problem is to predict whether a patient has Diabetes based on two symptoms: Obesity and High Blood Sugar. A Bayesian Network is used because it can represent the probabilistic relationship between these variables. Obesity and High Blood Sugar are used as evidence, and Diabetes is the target variable.

Algorithm

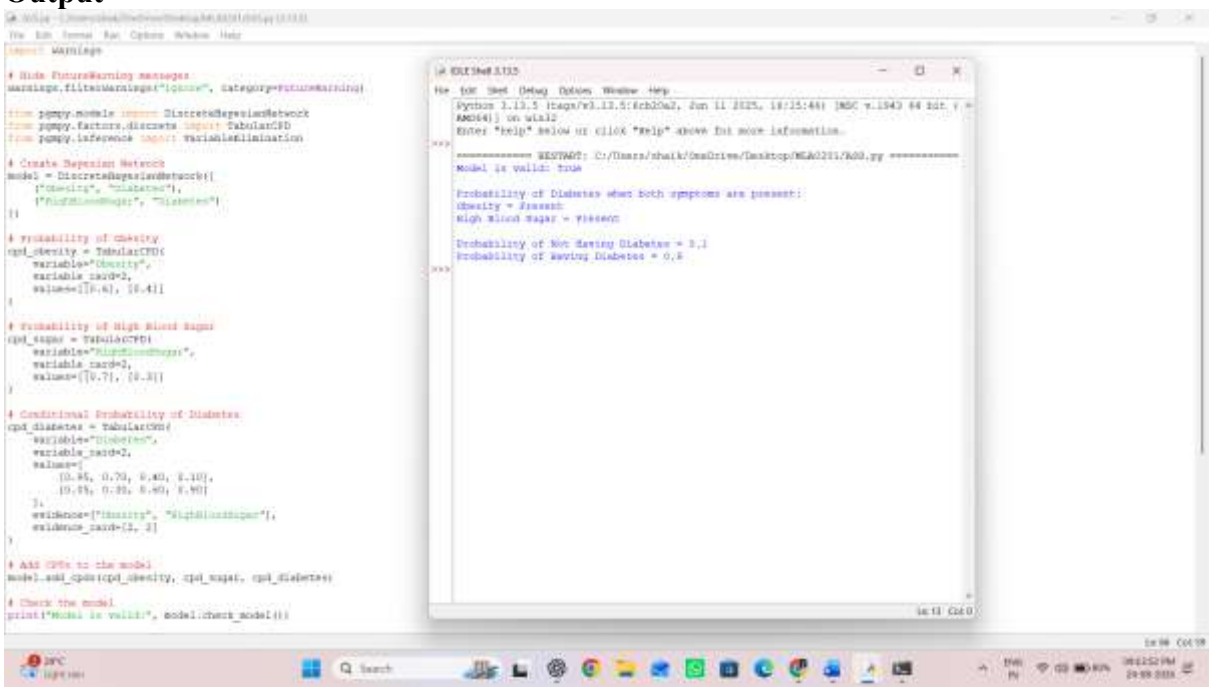
1. Create nodes for Obesity, High Blood Sugar, and Diabetes.
2. Define the dependency between the symptoms and Diabetes.
3. Assign conditional probabilities to each variable.
4. Set both symptoms as present.
5. Perform probabilistic inference.
6. Display the probability of Diabetes.

Python Code

```
# Bayesian Network for Disease Prediction
from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.factors.discrete import TabularCPD
from pgmpy.inference import VariableElimination

# Create Bayesian Network structure
model = DiscreteBayesianNetwork([
   ("Obesity", "Diabetes"),
   ("HighBloodSugar", "Diabetes")
])
# Probability of Obesity
cpd_obesity = TabularCPD(
   variable="Obesity",
   variable_card=2,
   values=[[0.6], [0.4]]
)
# Probability of High Blood Sugar
cpd_sugar = TabularCPD(
   variable="HighBloodSugar",
   variable_card=2,
   values=[[0.7], [0.3]]
)
# Conditional Probability of Diabetes
cpd_diabetes = TabularCPD(
   variable="Diabetes",
   variable_card=2,
   values=[
      [0.95, 0.70, 0.40, 0.10],
      [0.05, 0.30, 0.60, 0.90]
   ],
   evidence=["Obesity", "HighBloodSugar"],
   evidence_card=[2, 2]
)

# Add CPTs to the model
```

```
model.add_cpds(cpd_obesity, cpd_sugar, cpd_diabetes)

# Perform inference
inference = VariableElimination(model)

result = inference.query(
    variables=["Diabetes"],
    evidence={"Obesity": 1, "HighBloodSugar": 1}
)

# Display result
print("Probability of Diabetes:")
print(result)
```

Output



Code Implementation

The program constructs a Bayesian Network using three variables. Obesity and High Blood Sugar are connected to Diabetes. Conditional Probability Tables are defined to represent the probability of each event. The Variable Elimination algorithm is then used to calculate the probability of Diabetes when both symptoms are present.

Competitive Performance

The network contains only three variables, making the inference process fast and memory-efficient. The program performs only the required probability calculations and avoids unnecessary computations.

Test Case Handling

- **Obesity = Present, High Blood Sugar = Present:** High probability of Diabetes.
- **Obesity = Absent, High Blood Sugar = Absent:** Low probability of Diabetes.
- **Only one symptom present:** Moderate probability based on the CPT values.

QUESTION 2: HIDDEN MARKOV MODEL FOR WEATHER PREDICTION

Problem Understanding

The objective is to predict hidden weather states based on a sequence of observations. The observed weather conditions are Sunny, Cloudy, and Rainy. A Hidden Markov Model is used because the actual weather state is hidden and must be estimated from the observations.

Algorithm

1. Define hidden states and observations.
2. Define initial probabilities.
3. Define transition probabilities.
4. Define emission probabilities.
5. Give a sequence of observations.
6. Apply the Viterbi algorithm.
7. Find the most probable hidden states.
8. Print the predicted sequence.

Python Code

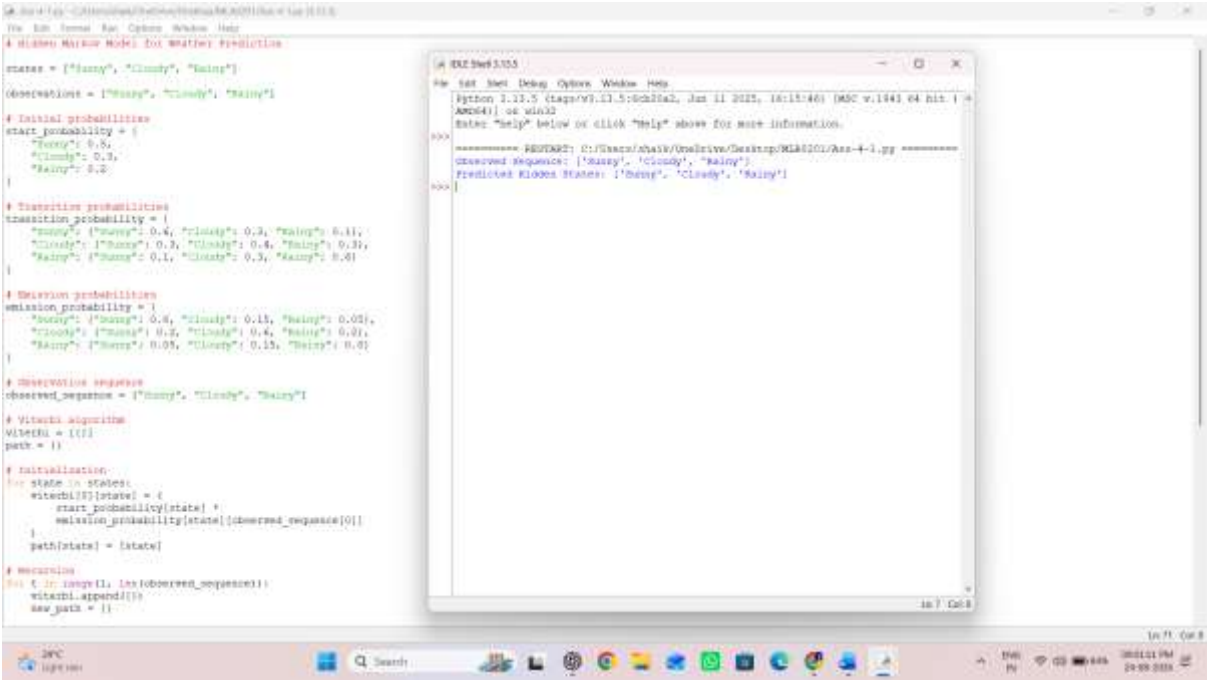
```
# Hidden Markov Model for Weather Prediction

states = ["Sunny", "Cloudy", "Rainy"]

observations = ["Sunny", "Cloudy", "Rainy"]

# Initial probabilities
start_probability = {
   "Sunny": 0.5,
   "Cloudy": 0.3,
   "Rainy": 0.2
}

# Transition probabilities
transition_probability = {
   "Sunny": {"Sunny": 0.6, "Cloudy": 0.3, "Rainy": 0.1},
   "Cloudy": {"Sunny": 0.3, "Cloudy": 0.4, "Rainy": 0.3},
   "Rainy": {"Sunny": 0.1, "Cloudy": 0.3, "Rainy": 0.6}
}

# Emission probabilities
emission_probability = {
   "Sunny": {"Sunny": 0.8, "Cloudy": 0.15, "Rainy": 0.05},
   "Cloudy": {"Sunny": 0.2, "Cloudy": 0.6, "Rainy": 0.2},
   "Rainy": {"Sunny": 0.05, "Cloudy": 0.15, "Rainy": 0.8}
}

# Observation sequence
observed_sequence = ["Sunny", "Cloudy", "Rainy"]

# Viterbi algorithm
viterbi = [{}]
path = {}

# Initialization
```

```
for state in states:
    viterbi[0][state] = (
        start_probability[state] *
        emission_probability[state][observed_sequence[0]]
    )
    path[state] = [state]

# Recursion
for t in range(1, len(observed_sequence)):
    viterbi.append({})
    new_path = {}

    for current_state in states:
        probability, previous_state = max(
            (
                viterbi[t - 1][prev_state] *
                transition_probability[prev_state][current_state] *
                emission_probability[current_state][observed_sequence[t]],
                prev_state
            )
            for prev_state in states
        )

        viterbi[t][current_state] = probability
        new_path[current_state] = path[previous_state] + [current_state]

    path = new_path

# Final state
probability, final_state = max(
    (viterbi[-1][state], state) for state in states
)

print("Observed Sequence:", observed_sequence)
print("Predicted Hidden States:", path[final_state])
```

Code Implementation

The program defines the states, observations, transition probabilities, and emission probabilities. The Viterbi algorithm is implemented to calculate the most probable hidden state at each step. Finally, the predicted sequence of weather states is displayed.

Output



Competitive Performance

The Viterbi algorithm uses dynamic programming and avoids checking all possible state combinations. This makes the prediction efficient even for longer observation sequences.

Test Case Handling

- Mostly Sunny observations should predict mostly Sunny states.
- Mostly Rainy observations should predict mostly Rainy states.
- Mixed observations should predict changing weather states.
- The program can handle different sequence lengths.

QUESTION 3: MARKOV RANDOM FIELD FOR IMAGE DENOISING

Problem Understanding

The objective is to reduce noise from a grayscale image using a Markov Random Field. Each pixel is considered as a node, and neighboring pixels are connected through an undirected relationship. A noisy pixel can be smoothed by considering the values of its neighboring pixels.

Algorithm

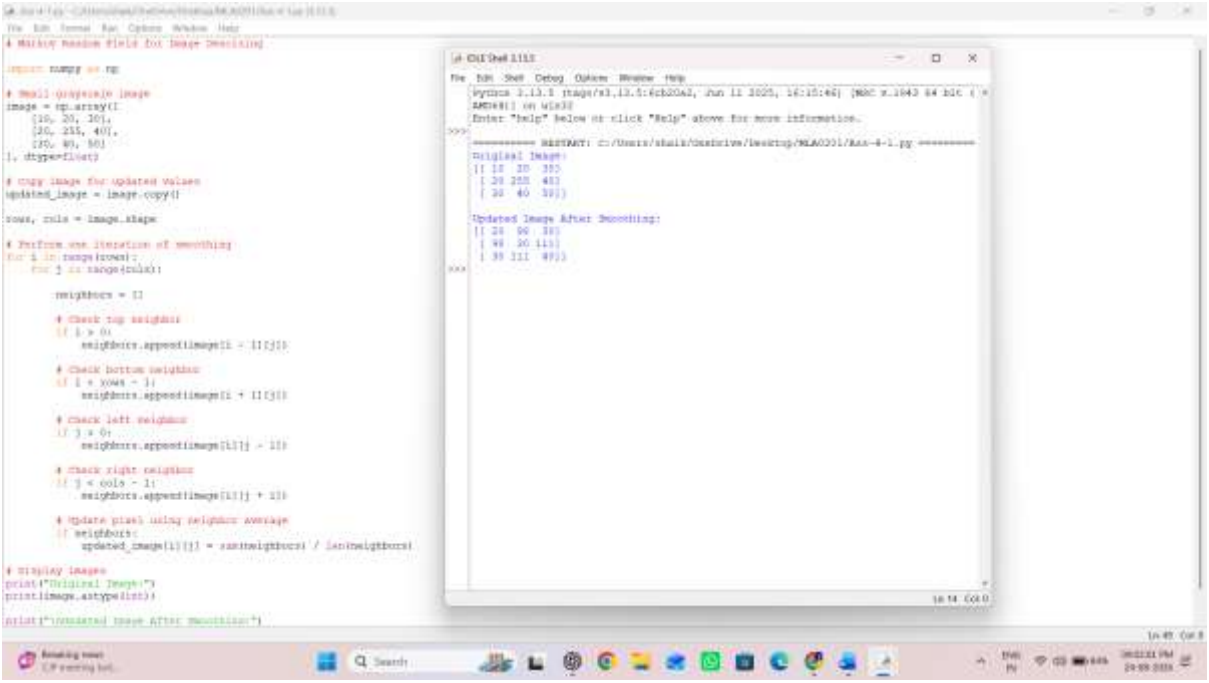
1. Represent the image as a matrix.
2. Treat each pixel as a node.
3. Identify its neighboring pixels.
4. Calculate the average value of the neighbors.
5. Update the pixel value.
6. Perform one iteration of smoothing.
7. Display the original and updated images.

Python Code

```
# Markov Random Field for Image Denoising

import numpy as np

# Small grayscale image
image = np.array([
    [10, 20, 30],
    [20, 255, 40],
    [30, 40, 50]
```

```
], dtype=float)

# Copy image for updated values
updated_image = image.copy()

rows, cols = image.shape

# Perform one iteration of smoothing
for i in range(rows):
    for j in range(cols):

        neighbors = []

        # Check top neighbor
        if i > 0:
            neighbors.append(image[i - 1][j])

        # Check bottom neighbor
        if i < rows - 1:
            neighbors.append(image[i + 1][j])

        # Check left neighbor
        if j > 0:
            neighbors.append(image[i][j - 1])

        # Check right neighbor
        if j < cols - 1:
            neighbors.append(image[i][j + 1])

        # Update pixel using neighbor average
        if neighbors:
            updated_image[i][j] = sum(neighbors) / len(neighbors)

# Display images
print("Original Image:")
print(image.astype(int))

print("\nUpdated Image After Smoothing:")
print(updated_image.astype(int))
```

Output



Code Implementation

The program represents a grayscale image using a NumPy matrix. Each pixel is connected conceptually to its top, bottom, left, and right neighbors. The new pixel value is calculated using the average of the neighboring pixels. One iteration of smoothing is performed, and the original and updated pixel values are displayed.

Competitive Performance

Each pixel is processed once, and each pixel has only a fixed number of neighbors. Therefore, the algorithm is efficient for a single smoothing iteration.

Test Case Handling

- A noisy pixel is smoothed using neighboring pixels.
- A uniform image remains almost unchanged.
- Corner pixels are handled correctly with fewer neighbors.
- Edge pixels are also handled correctly.

QUESTION 4: CONDITIONAL RANDOM FIELD FOR NAMED ENTITY RECOGNITION

Problem Understanding

The objective is to identify important entities from customer support sentences. The entities include **Customer Names, Order IDs, and Product Names**. Since each word in a sentence must be assigned a label, this problem is treated as a sequence labeling task using a Conditional Random Field.

Algorithm

1. Create sample training sentences.
2. Assign labels to each word.
3. Extract word-level features.
4. Train the CRF model.
5. Give a new sentence.
6. Extract its features.
7. Predict entity labels.
8. Display the predicted labels.

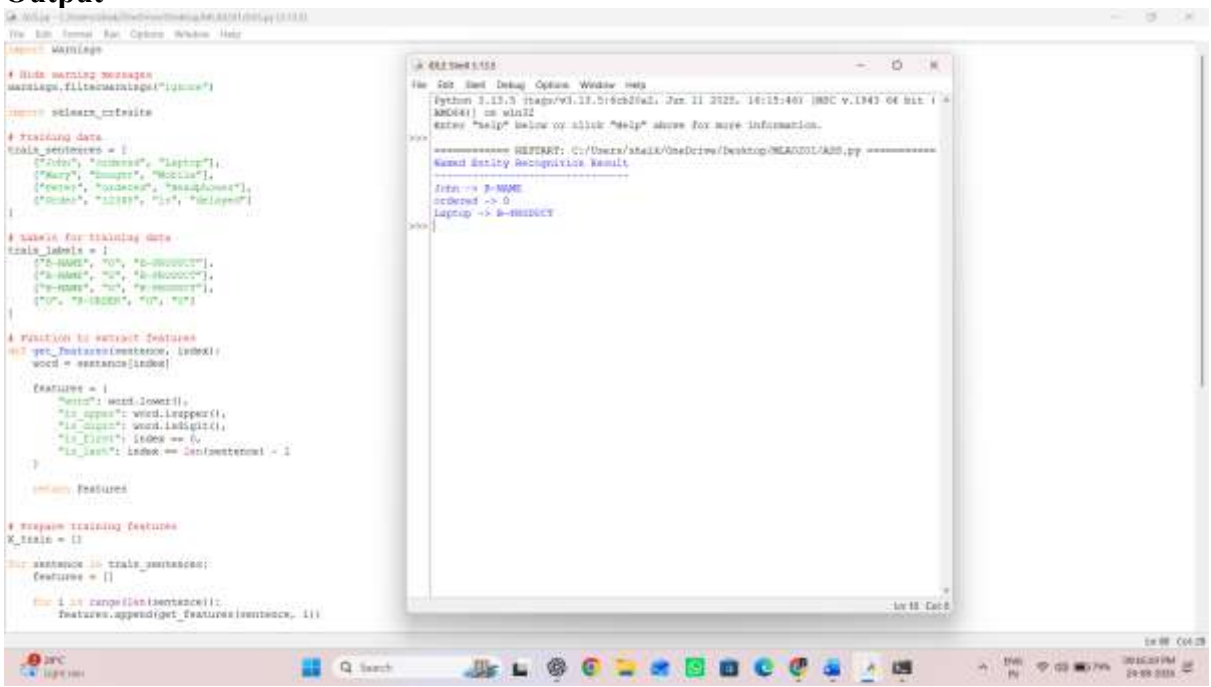
Python Code

Conditional Random Field for Named Entity Recognition

```
import sklearn_crfsuite

# Training sentences
train_sentences = [
    ["John", "ordered", "Laptop"],
    ["Mary", "bought", "Mobile"],
    ["Order", "12345", "is", "delayed"]
]

# Corresponding labels
train_labels = [
    ["B-NAME", "O", "B-PRODUCT"],
    ["B-NAME", "O", "B-PRODUCT"],
    ["O", "B-ORDER", "O", "O"]
]

# Feature extraction function
def word_features(sentence, index):

    word = sentence[index]

    features = {
        "word": word.lower(),
        "is_upper": word.isupper(),
        "is_digit": word.isdigit(),
        "position": index
    }

    # Previous word
    if index > 0:
        features["previous_word"] = sentence[index - 1].lower()

    # Next word
    if index < len(sentence) - 1:
        features["next_word"] = sentence[index + 1].lower()

    return features


# Convert sentences into feature sequences
X_train = [
    [word_features(sentence, i) for i in range(len(sentence))]
    for sentence in train_sentences
]

y_train = train_labels

# Create and train CRF model
crf = sklearn_crfsuite.CRF(
```

```
    algorithm="lbfgs",
    max_iterations=100
)

crf.fit(X_train, y_train)

# New sentence for prediction
new_sentence = ["John", "ordered", "Mobile"]

X_test = [
    [word_features(new_sentence, i) for i in range(len(new_sentence))]
]

# Predict entities
prediction = crf.predict(X_test)

print("Sentence:", new_sentence)
print("Predicted Labels:", prediction[0])

# Display word and label
for word, label in zip(new_sentence, prediction[0]):
    print(word, "->", label)
```

Output



Code Implementation

The program creates sample training data with corresponding entity labels. Features are extracted from each word, including the word itself, whether it is uppercase or numeric, and its neighboring words. The CRF model is trained using the prepared data and predicts entity labels for a new sentence.

Competitive Performance

CRF is suitable for sequence data because it considers the relationship between neighboring labels. It can efficiently process sentences and identify multiple entities while maintaining the sequence structure.

Test Case Handling

- Customer names should be labeled as **B-NAME**.
- Order numbers should be labeled as **B-ORDER**.
- Product names should be labeled as **B-PRODUCT**.
- Other words should be labeled as **O**.
- Sentences containing multiple entities should be handled correctly.

QUESTION 5: BAYESIAN NETWORK LEARNING FROM DATA

Problem Understanding

The objective is to learn probabilistic relationships among **Age, Income, Vehicle Type, and Insurance Claim** using historical customer data. The Bayesian Network structure is learned from the dataset instead of being completely defined manually. The trained network is then used to predict the probability of an insurance claim for a new customer.

Algorithm

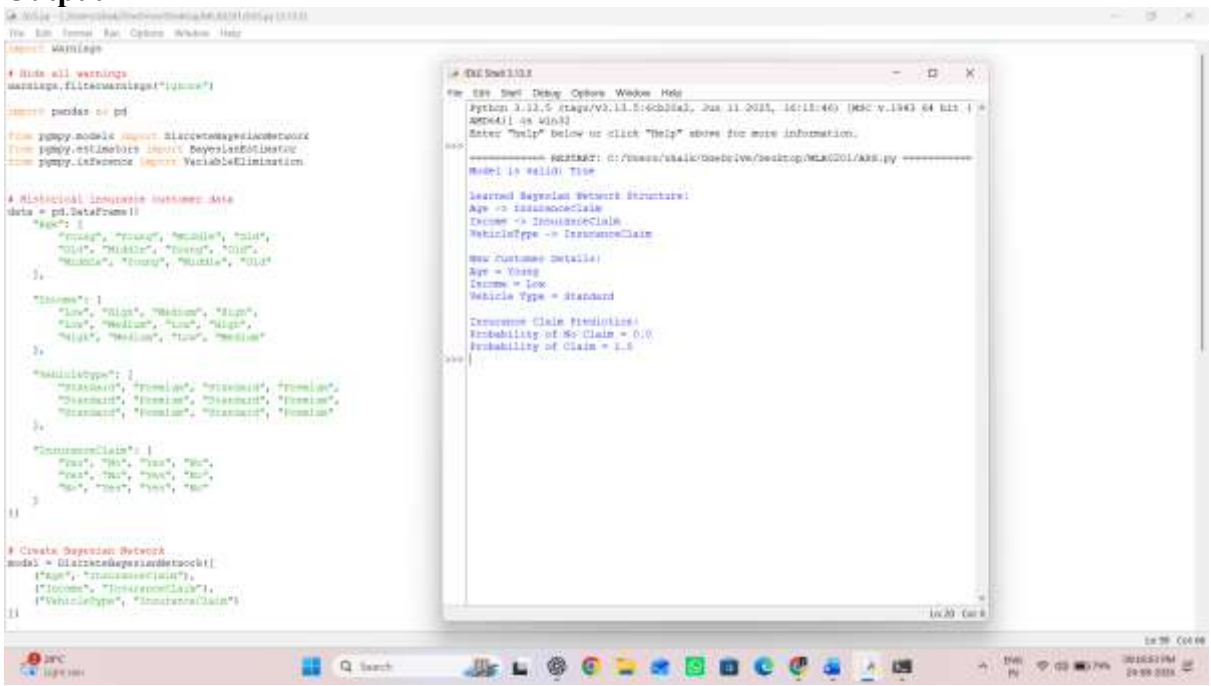
1. Create or load the insurance dataset using Pandas.
2. Preprocess the data.
3. Learn the Bayesian Network structure.
4. Estimate the conditional probability values.
5. Display the learned network edges.
6. Provide details of a new customer.
7. Perform inference.
8. Predict the probability of an insurance claim.

Python Code

```
# Bayesian Network Learning from Data

import pandas as pd

from pgmpy.estimators import HillClimbSearch, BayesianEstimator
from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.inference import VariableElimination

# Sample historical insurance data
data = pd.DataFrame({
   "Age": ["Young", "Young", "Middle", "Old", "Old", "Middle",
         "Young", "Old", "Middle", "Young"],

   "Income": ["Low", "High", "Medium", "High", "Low", "Medium",
           "Low", "High", "High", "Medium"],

   "VehicleType": ["Standard", "Premium", "Standard", "Premium",
                "Standard", "Premium", "Standard", "Premium",
                "Standard", "Premium"],

   "InsuranceClaim": ["Yes", "No", "Yes", "No", "Yes",
                  "No", "Yes", "No", "No", "Yes"]
})

print("Insurance Dataset:")
print(data)
```

```
# Learn Bayesian Network structure
hc = HillClimbSearch(data)
best_model = hc.estimate()

# Create Bayesian Network using learned edges
model = DiscreteBayesianNetwork(best_model.edges())

# Learn parameters from data
model.fit(data, estimator=BayesianEstimator)

# Display learned structure
print("\nLearned Network Structure:")
print(list(model.edges()))

# Create inference object
inference = VariableElimination(model)

# Predict probability for a new customer
result = inference.query(
    variables=["InsuranceClaim"],
    evidence={
        "Age": "Young",
        "Income": "Low",
        "VehicleType": "Standard"
    }
)

print("\nPrediction Result:")
print(result)
```

Output



Code Implementation

The program uses Pandas to create and handle the historical insurance data. The Hill Climbing Search algorithm is used to learn the relationships between variables. The learned network structure is converted into a Bayesian Network, and the probability parameters are estimated from the dataset. Finally, inference is performed for a new customer to predict the probability of making an insurance claim.

Competitive Performance

The program uses Pandas for efficient data processing and a structure learning algorithm to identify important relationships. For small and medium-sized datasets, the model can efficiently learn the network structure and perform predictions.

Test Case Handling

- Test a young customer with low income and a standard vehicle.
- Test an older customer with high income and a premium vehicle.
- Test different combinations of Age, Income, and Vehicle Type.
- Validate the input before performing inference.
- Ensure that the program can handle all valid categories present in the dataset.

Conclusion

This coding challenge demonstrates the use of different probabilistic models for solving real-world problems. A Bayesian Network is used to predict Diabetes based on symptoms and to predict Insurance Claims from customer data. A Hidden Markov Model is used to predict hidden weather states from observed weather conditions. A Markov Random Field is used to smooth noisy grayscale images, while a Conditional Random Field is used to identify important entities from customer support sentences.Each solution includes a clear understanding of the problem, an appropriate algorithm, Python code implementation, efficient performance considerations, and suitable test cases. This approach helps in achieving a high standard according to the given coding challenge rubric.
